



UNIVERSIDADE DO MINDELO
São Vicente – República de Cabo Verde

CURSO DE LICENCIATURA EM:
Engenharia Informática e Sistemas Computacionais - 3º ANO

RELATÓRIO TÉCNICO

TEMA:

Escalabilidade em Sistemas Distribuídos

Élvin Pires N° 5723

Professor:

Frederico Soares

Conteúdo

1. Introdução	3
2. Fundamentação Teórica	3
2.1 Conceito de Escalabilidade	3
2.2 Load Balancing (Balanceamento de Carga)	3
2.3 Replicação de Serviços	4
2.4 Stateless vs Stateful	4
2.5 Bottlenecks em Sistemas Distribuídos	4
3. Implementação Prática	5
3.1 Parte A — Escalabilidade do Monólito	5
3.2 Parte B — Escalabilidade dos Microserviços	5
3.3 Parte C — Testes de Carga	5
4. Resultados dos Testes de Carga	6
4.1 Resultados — Monólito com Load Balancer	6
4.2 Resultados — Microserviços com API Gateway	6
4.3 Comparação Direta dos Resultados	7
5. Análise Comparativa	8
5.1 Tabela Comparativa de Arquiteturas	8
5.2 Interpretação dos Resultados	8
5.3 Vantagens e Limitações Identificadas	9
6. Conclusão	10
7. Referências Bibliográficas	10

1. Introdução

Este trabalho dá continuidade ao Trabalho 01, no qual foram implementadas uma arquitetura monolítica e uma arquitetura de microserviços com API Gateway, aplicadas a um sistema de cardápio e pedidos para restaurante. O presente trabalho foca-se na escalabilidade dessas arquiteturas, explorando como cada uma responde ao aumento de carga de utilizadores.

A escalabilidade é uma propriedade fundamental em sistemas distribuídos modernos. À medida que o número de utilizadores cresce, o sistema deve ser capaz de manter tempos de resposta aceitáveis e garantir a disponibilidade do serviço. Para isso, recorre-se a técnicas como replicação de instâncias, balanceamento de carga e tolerância a falhas.

As questões centrais que este trabalho procura responder são:

- O que acontece quando o número de utilizadores aumenta simultaneamente?
- Como se pode escalar cada tipo de arquitetura?
- Quais são os limites e as vantagens de cada abordagem?

2. Fundamentação Teórica

2.1 Conceito de Escalabilidade

Escalabilidade refere-se à capacidade de um sistema de aumentar a sua capacidade de processamento em resposta ao crescimento da carga. Existem dois tipos principais:

- Escalabilidade Vertical (Scale Up): consiste em aumentar os recursos de uma única máquina, como adicionar mais CPU ou RAM. É simples, mas tem limites físicos e financeiros.
- Escalabilidade Horizontal (Scale Out): consiste em adicionar mais instâncias do serviço, distribuindo a carga entre várias máquinas. É a abordagem preferida em sistemas distribuídos modernos.

2.2 Load Balancing (Balanceamento de Carga)

O balanceamento de carga é o mecanismo responsável por distribuir os pedidos recebidos entre as várias instâncias disponíveis de um serviço. O algoritmo mais simples e eficaz para casos com instâncias homogêneas é o Round-Robin, que distribui os pedidos sequencialmente e de forma circular entre as instâncias disponíveis. Neste trabalho foi implementado um load balancer manual com Round-Robin tanto para o monólito como para os microserviços.

2.3 Replicação de Serviços

A replicação consiste em correr múltiplas cópias do mesmo serviço em paralelo. Em microserviços, é possível replicar apenas o serviço que está a ser o gargalo (bottleneck), sem necessidade de replicar toda a aplicação. No monólito, qualquer replicação implica duplicar toda a aplicação, incluindo partes que não necessitam de escala.

2.4 Stateless vs Stateful

Um serviço stateless não guarda estado entre pedidos — cada pedido é independente. Um serviço stateful guarda estado internamente (como listas em memória). Os serviços implementados neste trabalho são stateful, pois guardam dados em listas Python na memória de cada instância. Isto constitui uma limitação importante: quando se correm múltiplas instâncias, cada uma tem os seus próprios dados, o que pode causar inconsistências. Numa aplicação de produção, seria necessária uma base de dados centralizada e partilhada (como PostgreSQL ou Redis).

2.5 Bottlenecks em Sistemas Distribuídos

Um bottleneck é o componente que limita o desempenho global do sistema. Em microserviços, é possível identificar e escalar apenas o serviço que está a ser o gargalo. No monólito, como todos os componentes estão juntos, é necessário replicar tudo mesmo que apenas uma parte esteja sob pressão.

3. Implementação Prática

3.1 Parte A — Escalabilidade do Monólito

O servidor monolítico (`server.py`) foi modificado para aceitar a porta como argumento na linha de comandos, permitindo correr múltiplas instâncias na mesma máquina em portas diferentes. Foi também criado um load balancer dedicado (`balanceador_carga.py`) que corre na porta 8000 e distribui os pedidos entre as instâncias do monólito utilizando o algoritmo Round-Robin com a flag `threaded=True` para suportar pedidos concorrentes.

Configuração de execução do monólito escalado:

- Terminal 1: instância do monólito na porta 4001
- Terminal 2: instância do monólito na porta 4002
- Terminal 3: load balancer na porta 8000 (round-robin entre 4001 e 4002)

Limitação identificada: como cada instância do monólito guarda os seus dados em memória de forma independente, os dados não são partilhados entre instâncias. Um item criado na instância 4001 não estará disponível na instância 4002.

3.2 Parte B — Escalabilidade dos Microserviços

No caso dos microserviços, foi replicado especificamente o Serviço A (serviço de itens). O Serviço B (pedidos) manteve-se com uma única instância. O API Gateway foi atualizado para incluir lógica de Round-Robin entre as duas instâncias do Serviço A, e também configurado com `threaded=True`.

Configuração de execução dos microserviços escalados:

- Terminal 1: Serviço A, instância 1 na porta 3001
- Terminal 2: Serviço A, instância 2 na porta 3003
- Terminal 3: Serviço B na porta 3002
- Terminal 4: API Gateway na porta 5000 (com Round-Robin para o Serviço A)

3.3 Parte C — Testes de Carga

Foi criado um script de teste de carga (`teste_carga.py`) utilizando a biblioteca `threading` do Python. O script simula múltiplos utilizadores a fazer pedidos em simultâneo, medindo o tempo de resposta de cada pedido, o número de sucessos e erros, e o tempo total da simulação. Os testes foram executados com 50 utilizadores simultâneos para cada arquitetura, de forma independente, garantindo que cada sistema era testado com todos os seus serviços em funcionamento.

4. Resultados dos Testes de Carga

4.1 Resultados — Monólito com Load Balancer

Métrica	Resultado
Total de pedidos	50
Sucessos	50
Erros	0
Taxa de sucesso	100%
Tempo total	6.09 segundos
Tempo médio por pedido	5188.45 ms
Tempo mínimo	4178.39 ms
Tempo máximo	5916.14 ms

O monólito com load balancer respondeu corretamente a todos os 50 pedidos simultâneos sem nenhum erro. O round-robin foi verificado nos logs do balanceador, que mostrou a alternância perfeita entre as instâncias nas portas 4001 e 4002. O uso de `threaded=True` foi determinante para alcançar 100% de sucesso — sem esta configuração, pedidos concorrentes causavam timeouts.

4.2 Resultados — Microserviços com API Gateway

Métrica	Resultado
Total de pedidos	50
Sucessos	50
Erros	0
Taxa de sucesso	100%
Tempo total	6.86 segundos
Tempo médio por pedido	5696.00 ms
Tempo mínimo	4337.00 ms
Tempo máximo	6693.00 ms

Os microserviços com API Gateway também responderam corretamente a todos os 50 pedidos simultâneos sem nenhum erro. O load balancer no gateway distribuiu os pedidos

entre as duas instâncias do Serviço A. O tempo total ligeiramente superior ao do monólito (6.86s vs 6.09s) explica-se pela latência adicional introduzida pela comunicação HTTP entre o gateway e os serviços internos.

4.3 Comparação Direta dos Resultados

Métrica	Monólito + LB	Microserviços + GW	Diferença
Total de pedidos	50	50	—
Sucessos	50	50	Empate
Erros	0	0	Empate
Taxa de sucesso	100%	100%	Empate
Tempo total	6.09s	6.86s	+0.77s nos micro
Tempo médio	5188 ms	5696 ms	+508 ms nos micro
Tempo mínimo	4178 ms	4337 ms	+159 ms nos micro
Tempo máximo	5916 ms	6693 ms	+777 ms nos micro

5. Análise Comparativa

5.1 Tabela Comparativa de Arquiteturas

Critério	Monólito	Microserviços
Escalabilidade	Replica toda a app	Replica só o que precisa
Complexidade de gestão	3 processos	7 processos
Latência interna	Sem latência de rede	Latência HTTP entre serviços
Resiliência	Falha derruba tudo	Falha isolada por serviço
Consistência de dados	Problema ao escalar	Mesmo problema (sem BD partilhada)
Identificar bottleneck	Difícil (tudo junto)	Fácil (serviço específico)
Taxa de sucesso sob carga	100% (50/50)	100% (50/50)
Tempo médio de resposta	5188 ms	5696 ms

5.2 Interpretação dos Resultados

Ambas as arquiteturas alcançaram 100% de taxa de sucesso com 50 utilizadores simultâneos, o que demonstra que as duas abordagens de escalabilidade implementadas funcionaram corretamente. A diferença mais relevante é o tempo médio de resposta: o monólito foi cerca de 508 ms mais rápido por pedido (5188 ms vs 5696 ms).

Esta diferença de desempenho é esperada e explica-se pela natureza das arquiteturas. No monólito, o balanceador encaminha o pedido diretamente para o servidor — uma única chamada HTTP. Nos microserviços, cada pedido passa pelo API Gateway e depois é reencaminhado para o Serviço A — duas chamadas HTTP em sequência, o que introduz latência adicional de rede.

Os tempos absolutos (em torno de 5-6 segundos por pedido) são elevados para uma aplicação local, o que se explica pelo facto de o Flask correr em modo de desenvolvimento e de 50 threads concorrentes disputarem os mesmos recursos numa única máquina. Em produção, com um servidor WSGI como Gunicorn e múltiplos workers, estes tempos seriam significativamente inferiores.

5.3 Vantagens e Limitações Identificadas

Vantagens do Monólito escalado:

- Menor latência de resposta — sem camadas intermédias de comunicação HTTP.
- Menor complexidade operacional — apenas 3 processos necessários.
- Mais simples de depurar — tudo está no mesmo processo.

Vantagens dos Microserviços escalados:

- Escalabilidade seletiva — foi possível replicar apenas o Serviço A sem tocar no Serviço B.
- Resiliência — se o Serviço A falhar, o Serviço B continua a funcionar de forma independente.
- Flexibilidade de tecnologia — cada serviço pode ser desenvolvido e atualizado de forma independente.

Limitações comuns a ambas as arquiteturas:

- Estado em memória: a maior limitação identificada é que os dados são guardados em listas Python em memória. Com múltiplas instâncias, cada uma tem os seus próprios dados independentes. A solução para escalar de forma consistente seria uma base de dados centralizada partilhada.
- Flask em modo de desenvolvimento: o servidor Flask não é adequado para produção. O uso de `threaded=True` foi necessário para suportar pedidos concorrentes, mas num ambiente real usaria Gunicorn com múltiplos workers.
- Load balancer sem health check: o load balancer implementado não verifica se as instâncias estão ativas antes de reencaminhar pedidos.

6. Conclusão

Este trabalho permitiu explorar de forma prática os conceitos de escalabilidade horizontal, balanceamento de carga e replicação de serviços em dois tipos de arquitetura: monolítica e microserviços.

Os resultados dos testes de carga foram claros: ambas as arquiteturas conseguiram responder a 100% dos 50 pedidos simultâneos, demonstrando que as estratégias de escalabilidade implementadas funcionaram corretamente. O monólito apresentou um tempo médio de resposta ligeiramente inferior (5188 ms vs 5696 ms), o que se explica pela ausência de camadas de comunicação intermédias.

A principal vantagem dos microserviços revelou-se não na performance bruta, mas na flexibilidade de escala: foi possível replicar apenas o Serviço A (o componente sob maior pressão) sem necessidade de replicar o Serviço B ou qualquer outro componente. Esta escalabilidade seletiva é uma das razões pelas quais os microserviços são preferidos em sistemas de grande escala.

A limitação mais crítica identificada em ambas as arquiteturas é o estado em memória. Para escalar de forma verdadeiramente eficaz e consistente, os serviços teriam de ser stateless, com os dados guardados numa base de dados centralizada e partilhada por todas as instâncias.

Em suma, para sistemas com baixo volume de tráfego, o monólito é mais simples, mais rápido e mais fácil de gerir. Para sistemas com crescimento de utilizadores e necessidade de escalabilidade seletiva e resiliência, os microserviços são a escolha mais adequada, desde que acompanhados de uma infraestrutura de dados e orquestração adequadas.

7. Referências Bibliográficas

- Flask Documentation. <https://flask.palletsprojects.com>
- Python Threading Documentation. <https://docs.python.org/3/library/threading.html>